CosmoQ Cinema: Mobile Ticket Booking with Flutter and ASP.NET Core

Nguyen Nguyen Ngu (QE190064), Dao Thi Thanh Huong (QE190046), Pham Ngo Bich Tram (QE1900100)
Nguyen Hoang Ngoc Chau (QE190088), Nguyen Xuan Dang Khoa (QE1900126)
Student Developers, Class: SEK19B.Net
Department of Software Engineering
FPT University Quy Nhon Campus, Binh Dinh, Vietnam
Huynh Vang — Project Advisor, Lecturer
Department of Software Engineering, FPT University Quy Nhon Campus
Binh Dinh, Vietnam | vangh@fe.edu.vn

***Abstract*—Mobile cinema booking must coordinate local screen state with server-owned reservations while users leave the application for payment and later return. CosmoQ Cinema addresses this workflow with a Flutter customer application and a shared ASP.NET Core 8 backend behind a YARP gateway. This report examines the mobile implementation using startup code, navigation, remote repositories, HTTP adapters, platform configuration, SQL, and tests. The client uses explicit session notifications, secure-storage calls, coordinated refresh-token renewal, movie/showtime discovery, periodically updated seat maps, booking history, and QR ticket presentation. The backend uses Entity Framework Core and Npgsql with PostgreSQL, server-priced ten-minute holds, PayOS verification, and RabbitMQ-mediated notification and refund processing. Mobile-specific paths include external checkout, application-resumption verification, ticket sharing, calendar URLs, and staff QR-scanner integration. The analysis distinguishes these source implementations from successful physical-device execution. Seventeen isolated booking-service tests and twenty-one gateway tests passed; backend builds succeeded with four MovieService nullable warnings. The Flutter suite passed 209 tests, with one informational analysis lint. Docker-backed integration was blocked, and native release, live merchant payment, and device acceptance were not executed. The report contributes an evidence-linked description of mobile state recovery and booking coordination without treating plugin dependencies, test doubles, or platform folders as proof of operational completion.**

***Index Terms*—Mobile ticket booking, Flutter, Dart, ASP.NET Core, JWT, PostgreSQL, PayOS, QR tickets.**

I. Introduction

Mobile cinema booking brings discovery, seat choice, payment, and ticket access into a small-screen interaction. The user may leave the application to pay, return after the operating system pauses it, or revisit a booking after a network failure. A client must preserve an understandable distinction between local selection, a server-held reservation, and a paid ticket. CosmoQ Cinema addresses these tasks through a Flutter customer application connected to a shared ASP.NET Core backend [1], [2].

This paper focuses on the mobile build of the customer entry point in `Cinema_Mobile`. The repository also builds browser applications, but their management workspace is not a principal topic here. Platform directories exist for Android and iOS, as well as desktop targets; directory presence is not evidence that every platform has been built, signed, deployed, or tested. The verified source supports discussing a cross-platform implementation while retaining those distinctions.

The mobile implementation integrates three principal engineering mechanisms:

- **Recoverable client state.** Route-driven screens, remote catalog data, secure-storage calls, and coordinated renewal support the customer session.
- **Server-owned booking state.** Seat availability, transactional holds, order pricing, and payment reconciliation remain backend responsibilities.
- **Mobile ticket interaction.** External checkout and resumption, booking history, QR presentation, sharing, and a staff check-in path complete the flow.

Notification read state, promotions, points, and reviews extend these workflows. The contribution is their implementation and integration, rather than a novel framework or a measured improvement over another cinema application.

A. Inspection Method and Claim Boundaries

Source revision `55642353331f` and database revision `e455d297c76b` define the inspected snapshot on October 28, 2026. Inspection traced startup, routes, repositories, HTTP adapters, storage, platform manifests, backend controllers/services, SQL constraints, tests, and CI configuration. Requirements documents were not treated as proof of finished behavior. The technology references explain APIs; repository citations identify CosmoQ's implementation decisions.

The report separates strong implementation evidence from successful device execution. Real HTTP repositories are selected by startup, while fake repositories are used in tests. A source-level PayOS integration is not evidence that a mobile customer completed a banking transaction. Likewise, use of a secure-storage package is not a claim that storage and permissions were validated on every supported operating system. Section VII describes what was actually compiled, executed, or blocked.

TABLE I
VERIFIED MOBILE TECHNOLOGY STACK

Component	Technology	Responsibility
Mobile UI	Flutter, Dart, Material	Customer screens and navigation
Networking	Dart `http`	JSON requests and error handling
Session storage	`flutter_secure_storage`	Persist tokens and user data
Platform actions	`url_launcher`, `share_plus`	Checkout, links, sharing
Tickets	`qr_flutter`, `mobile_scanner`	QR display and staff scanning
Shared backend	ASP.NET Core 8; YARP	Identity, catalog, booking APIs
Data access	EF Core 8 and Npgsql	Six PostgreSQL databases
Events/payment	RabbitMQ; PayOS	Async follow-up and checkout

II. SYSTEM ARCHITECTURE OVERVIEW

A. Mobile-to-Service Topology

The mobile application uses the same public API gateway as the other client builds. It serializes requests as JSON and adds a bearer token for protected operations. The gateway forwards requests to AuthService, MovieService, or BookingService. Backend dependencies include PostgreSQL and RabbitMQ; external communication includes PayOS HTTP calls and SMTP email. Figure 1 shows these boundaries from the mobile customer's perspective [1], [3], [4].

There are three domain-service executables plus the gateway. MovieService also contains showtime and room code; BookingService contains payment and notification code. These modules are not separate deployed services merely because they have their own classes or database contexts. The gateway and services are individually defined in Docker Compose, which provides a local service-based topology rather than evidence of an operational cloud platform.

B. Verified Technology Stack

Table I distinguishes client, backend, and integration technology. The client uses Material widgets and Dart's asynchronous futures; no active Bloc or Riverpod implementation was needed to explain its state flow. Flutter's architecture documentation is relevant context [5], but the classes actually instantiated by `AppDependencies.remote` determine CosmoQ's behavior [1].

The locally installed SDK reports Flutter 3.47.2 and Dart 3.13.2, matching the manifest's Dart range. CI requests Flutter 3.47.4. Backend targets are .NET 8 and the review host uses .NET SDK 9.0.305. These version facts describe the review and configuration; they do not assert a version for a published mobile release [1], [6].

C. Configuration and Data Flow

`ApiConfig` defaults Android's emulator connection to `http://10.0.2.2:5000`; other default targets use localhost. A physical phone requires an appropriate `API_BASE_URL` build definition and network access to the gateway. A payment return URL can also be overridden. Defaults for local development should not be read as HTTPS deployment evidence [1], [4].

The UI calls repository interfaces, whose remote implementations invoke service adapters and a shared HTTP client. JSON models hold movies, showtimes, seats, bookings, payments, notifications, and authentication data. The repositories combine data when a screen requires more than one backend resource: seat layout comes from MovieService and occupied IDs from BookingService. This is client-side composition of server results, not direct database access.

III. BACKEND DESIGN AND IMPLEMENTATION

A. Identity as a Mobile Session Dependency

AuthService supports email registration and OTP verification, login, refresh, logout, password recovery, and profile/password changes. BCrypt hashes passwords and OTP values; JWTs carry the user's identity and role claims. JWT signing is HMAC-SHA256. The refresh flow revokes an accepted old refresh token before returning new session credentials. Tokens originate in this service rather than in a social-login or federated identity provider [7].

For the phone, the useful contract is an authentication response followed by authenticated API requests and recovery from expiry. Issuer, audience, signature, and lifetime checks occur in gateway/service JWT validation. Gateway middleware also requests current account status for protected requests. A locked account or changed roles can therefore invalidate continued use through the gateway even while a token's timestamp is still valid. An unreachable status service results in 503, which the mobile client must present as a service failure rather than a successful action [3], [8].

B. Catalog and Seat Information

MovieService supplies browsing, details, genres, and showtime selection. The showtime implementation checks that selected seats belong to the showtime's room, the show is open, and its start time is still in the future. Sweetbox seats are validated as pairs when the partner is defined. These checks run server-side independently of the phone's selection widgets [9].

Seat pricing uses a format surcharge followed by a type surcharge. Normal, VIP, and sweetbox values are defined in helpers; the resulting seat prices are returned in server responses and reobtained for booking creation. The mobile display helps the user understand a total, while the backend supplies the authoritative amount. This limits trust in stale or manipulated client inputs without asserting that every possible pricing scenario was device-tested.

C. Reservation Lifecycle

On booking submission, BookingService validates the request, rejects repeated seat IDs, prices concessions from its menu, resolves promotions, and applies permitted loyalty redemption. Customer identity is taken from claims by the

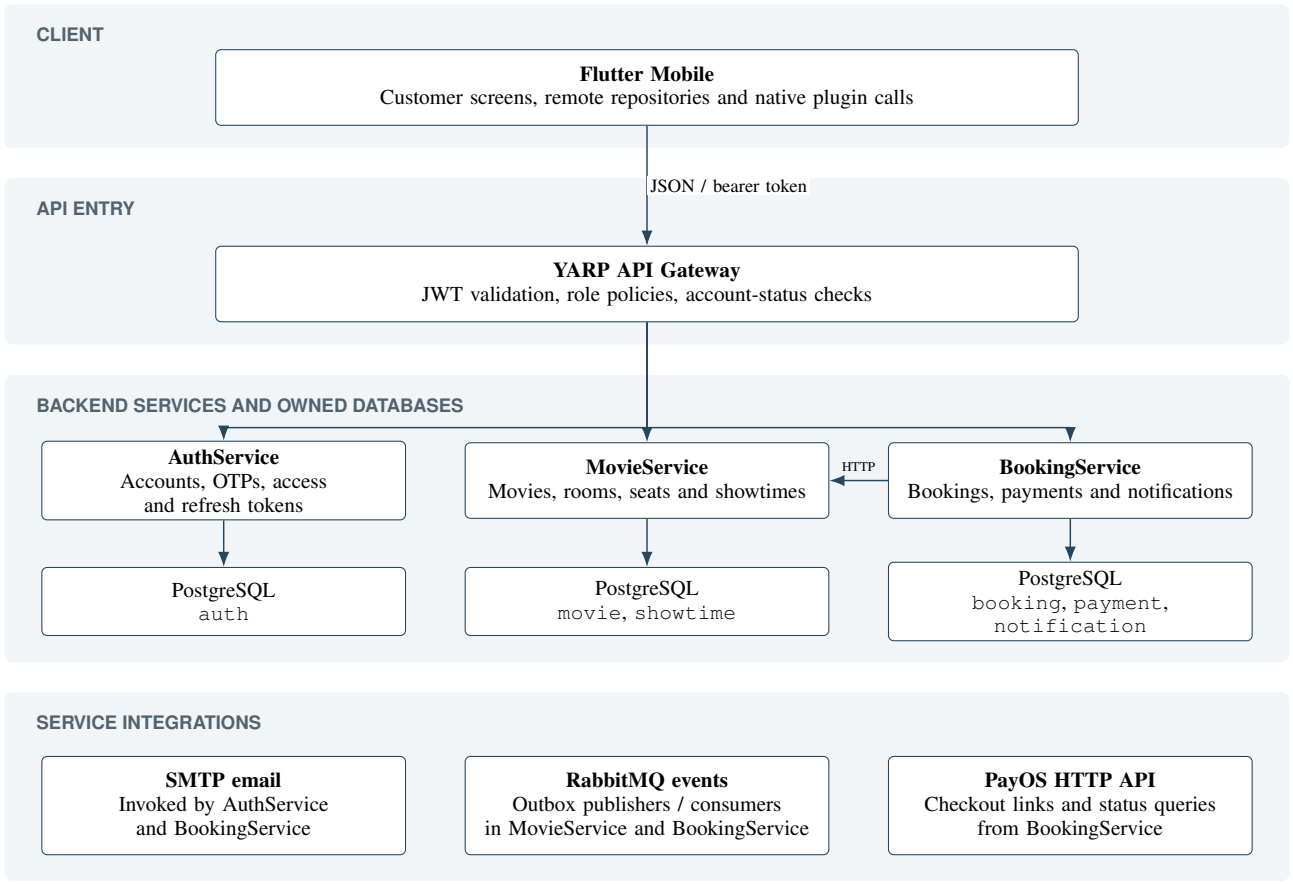


Fig. 1. Mobile system architecture: the app uses the shared gateway; backend services own persistence and integrations. Solid arrows show client/API and persistence paths. The integration cards identify the backend services responsible for each external binding. Database labels abbreviate the six `cinema_..._db` names; all six use one PostgreSQL instance in Compose.

controller. The service records the showtime time received from MovieService; movie title retrieval includes a request-title fallback if the remote lookup returns no title [2].

Reservation conflict boundary. The reservation transaction locks existing seat/showtime records with `FOR UPDATE`. It rejects booked seats and active holds, creates a pending booking with a ten-minute expiry, and associates held seats. The unique `(showtime_id, seat_id)` key handles competing inserts when a reservation row has not existed before. A unique-violation exception is translated into a conflict response. The phone cannot turn a locally highlighted seat into a reservation until this transaction succeeds [2], [10], [11].

Expiry and cancellation. A background expiry worker changes overdue pending bookings and releases their seat holds. Payment confirmation attempts to transition the booking to a paid state, while cancellation and refund paths have distinct rules. In particular, a paid customer cancellation request can become `REFUND_PENDING`; it does not simply free the seats before the refund decision. This distinction must remain visible in booking history rather than being reduced to a client-side Boolean [2].

D. Database Design for Mobile Workflows

SQL initialization creates six databases on the configured PostgreSQL 17 instance. Auth data uses users, roles, a join table, refresh tokens, and OTPs. Movie data is separate from rooms, seats, and showtimes. Booking data includes seat snapshots, reservations, concessions, promotions, loyalty records, and reviews. Payment and notification records have separate contexts/databases. The Npgsql EF Core provider connects these models to PostgreSQL [2], [10], [12].

The selected ERD in Fig. 2 distinguishes SQL foreign keys from logical ID references. Rooms have declared seat/showtime relationships; bookings have declared booking-seat relationships; refunds reference payments. Cross-database user, movie, and booking IDs are application references rather than SQL-enforced relationships. The reservation's booking ID is not declared as a foreign key in the inspected base schema. A full mobile workflow therefore depends on service validation and event processing as well as local constraints.

Schema files and migrations are maintained in the initialized database submodule, with additional backend SQL updates. The Compose initialization and migration jobs mount these sources. Inspection verifies intended schema structure and execution scripts, not successful migration of a running

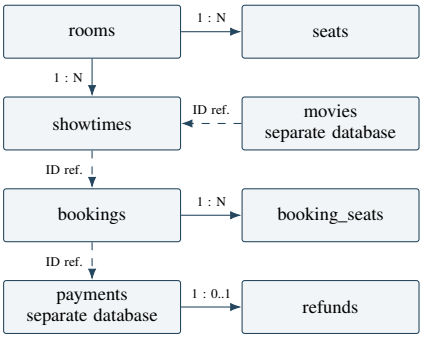


Fig. 2. Selected schema relationships reconstructed from SQL. Solid arrows denote parent-to-child SQL foreign keys; dashed arrows denote logical ID references. Reservations, identity, and other extension tables are omitted.

database. No live data or production credentials were required or inspected for the relational analysis [4], [10].

E. Payment and Asynchronous Follow-Up

PayOS integration requests checkout links and queries payment status. Creation includes merchant headers and an HMAC signature. The webhook path validates the payload signature before trusting fields and checks order/amount against the saved payment. A verified provider status can also trigger reconciliation when the customer returns to the app. These source paths avoid equating an external-browser launch with payment success [2], [13].

Payment and booking records are not described as one distributed transaction. The code contains a confirmation-or-refund path when payment succeeds after the booking can no longer be confirmed. Staff record refund processing and transfer references; automatic bank refund execution was not found. RabbitMQ outbox workers publish persistent events after local commits, and consumers process notification/refund and showtime-cancellation effects. Publisher confirms and event-ID deduplication support retries but do not establish exactly-once external email delivery [2], [14].

IV. Mobile Application Architecture and Implementation

A. Startup and Navigation

`main.dart` initializes Flutter bindings, creates the session object and remote dependencies, attempts session restoration, and wraps the application in `SessionProvider`. Unreadable storage is handled as a signed-out start. Named routes create screens with their dependencies; an authenticated route can redirect to sign-in with a pending destination [1].

The customer app exposes discovery, ticket history, notifications, and account functions. Detail screens accept route IDs and reload data when an earlier screen's argument is unavailable. This design also serves browser builds, but on mobile it gives a consistent navigation contract without relying on one global mutable booking object. It does not prove operating-system deep-link registration or recovery of every transient form after process termination. Table II lists representative route contracts.

TABLE II
Representative Mobile Screens

Screen	Named route	Responsibility
Home	`/home`	Catalog and movie filters
Movie details	`/movies/{id}`	Metadata and showtimes
Seat selection	`/showtimes/{id}/seats`	Room layout and local choice
Summary	`/showtimes/{id}/summary`	Order options and submission
My bookings	`/bookings`	Ticket history
Booking details	`/bookings/{id}`	QR and lifecycle state
Payment	`/bookings/{id}/payment`	External checkout and checks
Notifications	`/notifications`	Read/unread messages
Profile	`/profile`	Account and member functions

B. Session State and Token Recovery

`SessionState` is a `ChangeNotifier`, distributed through an `InheritedNotifier`. It stores guest/authenticated state, user information, expiration messages, and unread-notification count. Individual screens own selection, loading, error, and pagination state. The code therefore uses a small explicit state arrangement rather than an evidenced application-wide Bloc or Redux architecture [1].

Concurrent token renewal. The HTTP client gives requests a default fifteen-second timeout and decodes structured API failures. On a protected 401 it asks `TokenManager` to renew the token, then retries once. Listing 1 reproduces the shared-future mechanism. Since the server rotates refresh tokens, independently refreshing concurrent failed requests could otherwise reuse an invalidated token. Additional session-version checks stop an in-flight renewal from overriding a later account-expiration event [1].

```
return _pendingRefresh ??=
    _refresh(failedToken).whenComplete(() {
  _pendingRefresh = null;
});
```

Listing 1. Concurrent refresh coordination from TokenManager.

Visible session recovery. Network errors can preserve session data for a retry; rejected refresh credentials clear it. `SessionApp` responds to the expiration message by opening sign-in and displaying an explanatory notice. This links transport errors to navigation and prevents the UI from silently continuing with an expired account state. It is an implementation mechanism, not evidence that every race or device interruption has been exhaustively tested.

C. UI Composition and Feedback

Feature directories group authentication, movie, seat, booking, payment, notification, profile, and staff screens. Shared theme files define spacing, typography, color, and radius values. Reusable buttons, panels, loading, empty, and error states keep common interactions consistent. Screen code uses asynchronous calls, mounted checks, and busy flags to avoid

updating disposed widgets or repeatedly submitting an active action [1], [15].

The layout switches desktop navigation at 900 logical pixels, while phone layouts use the narrow application frame. Seat maps and reusable column components respond to available width rather than assuming a fixed desktop canvas. The existence of phone-sized widget assertions supports testing those layouts with fixtures; it is not a general accessibility, device-compatibility, or performance certification.

D. Platform-Specific Actions

Platform actions are isolated from the shared booking contract [1]:

- **External activities.** Native trailers and PayOS checkout launch through `url_launcher`; the native trailer path does not embed the Web player.
- **Sharing and calendar links.** `share_plus` invokes sharing, with a clipboard fallback. Calendar support constructs a Google Calendar event URL, not a synchronized device-calendar integration.
- **Ticket display and scanning.** `qr_flutter` renders tickets. A staff-only route uses ticket lookup or `mobile_scanner` before submitting server check-in.

The iOS property list declares a camera-purpose description and Android declares Internet access. These settings show platform intent; camera permissions, release packaging, and physical-device scanning remain unverified.

V. Core Business Workflows

A. Authentication and Account Use

A customer may browse as a guest, but protected booking/account actions require sign-in. Registration sends account data to AuthService and continues through email OTP verification. Password recovery and profile editing have remote service paths. Session restoration reads saved credentials and current-user information; failures requiring authentication return the user to sign-in. The application does not establish biometric login merely by depending on a package that can support biometric options [1], [7].

B. Discovery, Showtimes, and Seat Choice

Home browsing combines now-showing/coming-soon selection, genres, and title search, with a 350-millisecond debounce. Details expose metadata, trailer actions, reviews, and open showtimes. The catalog repository fetches schedules within a seven-day customer window. Once a show is chosen, room seats and occupied IDs are combined into `ShowtimeSeatsData` [1].

Local choice and server hold. The selected seat set remains local until order submission. A five-second timer refreshes the visible map; conflicting local seats are removed, with sweetbox partner handling. Failed refreshes retain the last map and retry on a later tick. The booking transaction is consequently essential even when the display recently looked current. This is periodically refreshed availability, not verified WebSocket or SignalR push [1], [2].

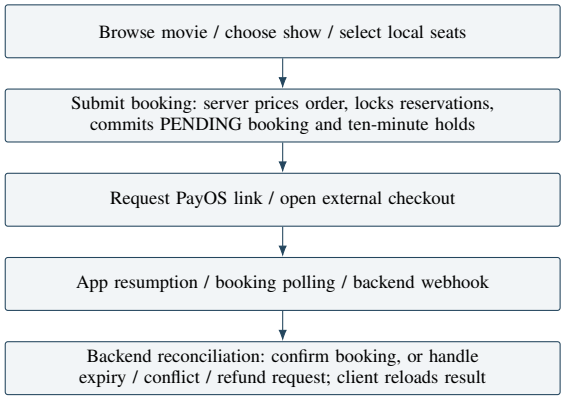


Fig. 3. Mobile external-checkout and resumption flow. No client redirect or timer is itself proof of payment.

C. Order Calculation

Summary collects seats, concessions, promotion choice, and requested member points before submission. The backend recalculates the total. If S is seat value, D the applicable ticket promotion, C concessions, and q redeemed points, the code implements

$$T = S - D + C - 1000q. \tag{1}$$

The redemption ceiling is 50% of the pre-points total and the account's available balance. Promotion reductions are capped and rounded down to 1,000-VND steps. This notation explains real arithmetic rather than adding an unrelated optimization model [2].

Loyalty earning is also explicit. For paid amount A and tier earn rate r, the helper returns $\lfloor Ar/1000 \rfloor$ points. The configured tier rates are 5%, 7%, and 10% for MEMBER, VIP, and VVIP. These values describe code constants; no customer-retention or revenue effect has been measured.

D. External Checkout and App Resumption

Figure 3 describes the mobile checkout path. After creation of a pending booking, the app requests a PayOS link and launches it externally. `PaymentScreen` observes application lifecycle changes: a resumed app that previously opened checkout requests verification. It also polls booking state every five seconds, with provider verification every sixth eligible poll, and disposes timers when leaving the screen [1].

Resumption and reconciliation. The displayed countdown is computed from the booking expiry; it does not authorize or extend the hold. A successful provider result is reconciled on the backend before a confirmed ticket is shown. If the application is terminated, booking history and ID-based detail loading provide a recovery path; no claim is made that the OS automatically returns to the exact prior payment screen or that offline purchase completion is supported.

E. Tickets, Notifications, and Reviews

Booking history/detail shows current status and paid-ticket QR content. Sharing produces ticket text and a booking code;

TABLE III
MAJOR MOBILE FUNCTIONAL MODULES

Module	Customer interaction	Supporting layer
Identity	Login, OTP, account recovery	Auth repository / AuthService
Discovery	Search, details, schedules	Catalog repository / MovieService
Reservation	Seats, options, summary	Booking repository / BookingService
Payment	External checkout, resumption	Payment screen / PayOS adapter
Tickets	History, QR, share, calendar URL	Flutter/native plugins
Messages	Lists, detail, read state	Notification APIs / SMTP handlers
Membership	Points, discounts, reviews	Booking business services

the QR must not be described as cryptographically signed unless separately evidenced. Staff check-in validates the booking on the server and records admission within a configured time window. Review submission requires an owned booking that has been checked in, with a unique user/movie review mapping [1], [2].

Notification lists retrieve stored messages and update read state through APIs; the home/session UI tracks an unread count. Backend workers and event handlers create reminder and ticket-related messages and send SMTP email. No Firebase Cloud Messaging, APNs registration, or background OS push delivery was established in the inspected implementation. Source-backed in-app records and email must therefore be distinguished from native push notifications.

VI. SECURITY AND DATA PROTECTION

A. Token and Account Protection

Protection is divided between client persistence and API enforcement:

- **Local credentials.** `SecureAuthTokenStorage` persists tokens, expiry, and user data using serialized writes and clears unreadable storage. Platform-specific package implementations are used [1], [16]; hardware backing and compromised-device resistance were not tested.
- **Server enforcement.** Password/OTP hashing, input validation, role checks, and resource ownership constrain API use. Customer identity comes from claims and pricing uses backend data [2], [7].
- **Gateway controls.** Rate limiting and current-account validation add request checks independently of the mobile screen state [3].

No additional biometric gate, certificate pinning, or device attestation was established. These mechanisms do not constitute a complete security guarantee.

B. Transport and External Data

The default API addresses are HTTP development endpoints. HTTPS operation, certificate configuration, and Android/iOS release network policies must be verified for a named deployment. The client does not embed merchant signing secrets in the inspected dependency construction; PayOS signing and webhook checks occur in the backend. Actual secret values were not opened or printed [1], [2], [4].

Ticket sharing intentionally exposes booking information to the recipient. Calendar actions construct an external URL containing event details. Those flows should be explained to users in a release assessment rather than being described as encrypted ticket distribution. Notification and payment records also contain personal or payer metadata; no retention-policy or deletion compliance guarantee can be derived solely from their entity definitions.

TABLE IV
VERIFICATION EXECUTED ON OCTOBER 28, 2026

Check and method	Observed result / status
BookingService isolated unit filter	**PASS**: 17 tests, zero failed
ApiGateway test project	**PASS**: 21 tests, zero failed
AuthService build	**PASS**: zero errors/warnings
MovieService build	**PASS**: zero errors; four nullable warnings
Flutter analysis	1 informational lint; no errors/warnings
Flutter unit/widget suite	**PASS**: 209 tests after package restore
PostgreSQL/RabbitMQ integration suites	**BLOCKED**: Docker daemon unavailable
SQL/migration execution	**NOT EXECUTED**: no running database
Live PayOS, SMTP, deployed acceptance	**NOT EXECUTED**: external services not exercised
Native release/device verification	**NOT EXECUTED**: no device acceptance run

VII. TESTING, VERIFICATION, AND EVALUATION

A. Executed Checks

Table IV records the review's verification outcomes. The installed Flutter/Dart SDK and .NET SDK were identified directly. Backend isolated tests succeeded: 17 BookingService unit cases and 21 gateway cases. AuthService compiled with no warnings; MovieService compiled with four nullable warnings. Gateway and BookingService builds also completed as part of their test runs. These results do not measure device startup, frame rates, battery use, or production API response times.

After restoring the existing declared Flutter dependencies, the full `flutter test -no-pub -reporter expanded` run passed 209 tests. `flutter analyze -no-pub` reported one informational finding, `curly_braces_in_flow_control_structures`, in `trailer_url.dart:12`; no analysis errors or warnings remained. The analyzer was therefore not a clean zero-finding run. The earlier no-restore attempt failed because locally cached QR/sharing packages were missing, and is not counted as a passed test run. No application source was changed to obtain the restored-package result. The shared client also

compiled for its two browser entry points; this does not substitute for Android or iOS release verification.

B. What the Test Suite Can Establish

Flutter tests cover model parsing, API-client handling, form validation, navigation, seat maps, checkout states, account changes, ticket features, and responsive views. They inject repositories, HTTP responses, and platform substitutes. This accords with the distinction between unit/widget tests and complete-device integration described by Flutter [6], [17]. Passing such tests does not demonstrate camera capture, an actual share sheet, a real secure-storage backend, or banking completion on a physical phone.

Backend integration fixtures create PostgreSQL through Testcontainers and can enable a RabbitMQ container; MovieService, PayOS, and SMTP are otherwise substituted in the booking fixture. The Docker daemon was unavailable, so those container suites and a full deployment were blocked. Database SQL and mappings were inspected, but migrations were not run against a live database. Android/iOS release builds and physical-device acceptance were not executed [6], [10].

C. Interpretation

The evidence supports an implemented remote-client architecture and specific tested backend helpers/policies. It does not support an overall mobile acceptance rate, production availability, a completed merchant transaction, or a quantified security level. A workflow with both UI code and API code is reported as implemented in source with runtime verification outstanding where appropriate. No fixture data or example payment record is counted as a successful real customer action.

VIII. Limitations and Future Development

A. Current Limitations

The client depends on connectivity for catalog retrieval, booking, and payment confirmation. A stale seat map is possible between polling ticks or after a failed refresh. External checkout introduces operating-system lifecycle and browser behavior that widget mocks cannot fully reproduce. The implementation does not establish offline booking, native push, universal app links, or biometric authentication merely through route names or dependencies [1].

Refund transfer remains manual staff work. Email sending and database updates are separate operations despite event-ID checks, so exactly-once SMTP delivery is not guaranteed. Local-time timestamps with Npgsql legacy behavior constrain assumptions about users in other time zones. The source includes a calendar duration fallback of 150 minutes when exact showtime data cannot be obtained; that fallback is a convenience, not guaranteed schedule accuracy [1], [2], [4].

The review could not demonstrate live provider operation, container integration, native secure storage, QR scanning, or signed store distribution. Existing SQL seed payment labels are demonstration data, not evidence of active VNPAY or MOMO implementations. Platform folders establish potential targets rather than a tested support matrix. These limits prevent describing the application as production-ready.

B. Recommendations

Recommended next steps are a controlled Android/iOS device matrix, release build and permission checks, and lifecycle tests that interrupt checkout at multiple stages. Verification should include token restoration after process death, network transitions, camera denial, share-sheet failure, and repeated payment reconciliation. Container suites and schema migrations should be run before a complete system acceptance test.

Further recommendations include validating HTTPS and release network policy, testing accessibility and text scaling, resolving backend nullable warnings, and reviewing notification delivery requirements before considering native push. Any push, offline cache, biometric gate, or deep-link enhancement should be treated as proposed development rather than an implemented feature or committed project roadmap.

IX. Conclusion

CosmoQ's mobile implementation provides a Flutter customer workflow over a shared ASP.NET Core backend. Session recovery, remote repositories, periodic seat updates, server-created holds, external checkout reconciliation, and QR ticket access form the central implementation. SQL constraints and backend authorization carry responsibilities that cannot safely be delegated to phone UI state. The reported verification establishes selected compilation and automated-test outcomes while preserving the gap to physical-device and live provider validation. The system is therefore documented as an implemented mobile client with explicit operational verification limits.

Acknowledgment

The student developers thank Mr. Huynh Vang, Department of Software Engineering, FPT University Quy Nhon Campus, for his guidance as project advisor and for supporting the team's software engineering work on CosmoQ Cinema.

References

[1] CosmoQ Cinema Student Developers, "Flutter client entry points, routing, session state, and API adapters," Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_Mobile/lib, inspected Oct. 28, 2026.

[2] ——, "Booking, payment, refund, loyalty, notification, and messaging implementation," Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_BE/BookingService/BookingService, inspected Oct. 28, 2026.

[3] ——, "Gateway policies, reverse proxy, and account-status middleware," Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_BE/ApiGateway/ApiGateway, inspected Oct. 28, 2026.

[4] ——, "Compose deployment and database initialization scripts," Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_BE, inspected Oct. 28, 2026.

[5] Flutter, "Architecting Flutter apps," https://docs.flutter.dev/app-architecture, accessed Oct. 28, 2026.

[6] CosmoQ Cinema Student Developers, “Backend tests and Flutter verification workflows,” Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/., inspected Oct. 28, 2026.
[7] ——, “Authentication, account validation, JWT generation, and user management,” Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_BE/AuthService/AuthService, inspected Oct. 28, 2026.
[8] Microsoft, “Configure JWT bearer authentication in ASP.NET Core,” https://learn.microsoft.com/en-us/aspnet/core/security/authentication/configure-jwt-bearer-authentication, accessed Oct. 28, 2026.
[9] CosmoQ Cinema Student Developers, “Movie, room, showtime, and seat-pricing implementation,” Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_BE/MovieService/MovieService, inspected Oct. 28, 2026.
[10] ——, “Cinema database SQL schemas and migrations,” Repository source, revision e455d297c76b, https://github.com/PRM393-Cinema/Project-Cinema-DB/tree/e455d297c76b14e3fd6aa8c0af4bb11f77ec6a2b, inspected Oct. 28, 2026.
[11] PostgreSQL Global Development Group, “PostgreSQL 17: Constraints,” https://www.postgresql.org/docs/17/ddl-constraints.html, accessed Oct. 28, 2026.
[12] Npgsql, “Npgsql Entity Framework Core provider,” https://www.npgsql.org/efcore/, accessed Oct. 28, 2026.
[13] payOS, “payOS API,” https://payos.vn/docs/api/, accessed Oct. 28, 2026.
[14] RabbitMQ, “Consumer acknowledgements and publisher confirms,” https://www.rabbitmq.com/docs/confirms, accessed Oct. 28, 2026.
[15] CosmoQ Cinema Student Developers, “Customer browser layout and administration workflows,” Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_Mobile/lib/features, inspected Oct. 28, 2026.
[16] flutter_secure_storage Maintainers, “flutter_secure_storage 11.2.0,” https://pub.dev/packages/flutter_secure_storage, accessed Oct. 28, 2026.
[17] Flutter, “Testing Flutter apps,” https://docs.flutter.dev/testing/overview, accessed Oct. 28, 2026.